

1st Assignment on Neural Networks Deep Learning

Florian Bylykbashi
Department of Electrical and Computer Engineering
AEM: 11012

August 16, 2026

Introduction - General Information

For the midterm assignment, we used Visual Studio Code where we were asked to use two machine learning algorithms, K-NearestNeighbor and NearestCentroid, to categorize a dataset of our choice. To achieve this, it was necessary to create a .venv environment so that the appropriate libraries could be installed via pip install.

For the main assignment, we used Google Colab because it was necessary to use the T4 GPU provided by Google, as our models needed a considerable amount of time to train. For this assignment, almost 10 different models have been implemented in which each time we change something to see how our network will be affected (how well it will train, what accuracy it will have). At the end, there are three categorization examples.

The libraries that have been used are tensorflow, numpy, time, sklearn, and matplotlib.pyplot.

From the given datasets, Cifar10 was chosen, which is a collection of 60,000 32x32 images divided into 10 different classes. These classes are:

1. Airplane
2. Automobile
3. Bird
4. Cat
5. Deer
6. Dog
7. Frog
8. Horse
9. Ship
10. Truck

Finally, it is divided into a training set of 50,000 images and a testing set of 10,000 images. For the purposes of this assignment, we use a **validation set** of 10,000 images and only 40,000 images from the training set.

Below begins the analysis, first of the two classifiers, and then the analysis of each model separately. At the end, there will be comments and conclusions regarding our results.

1-NearestNeighbor

This classifier for $K=1$ takes an image that it wants to classify. It then calculates its Euclidean distance from all the images in our training set. From these distances, it then keeps the one that is closest to our image. Thus, our image then takes the class of the closest image (e.g., if the closest image had the class 'cat', then ours will also take the same).

In our code, we first load our dataset via tensorflow. Then we have the option to either use our entire dataset or just a part of it. In our example, we use the whole dataset, but the user has the option to reduce the samples to see the changes in accuracy. Because the algorithm does not accept 2D images, we flatten the images into a large vector of 3072 elements ($32 \times 32 \times 3$). Then, through the auxiliary commands of our libraries, training and evaluation take place, and at the end, the results are printed.

3-NearestNeighbor

Similar to before, the classifier takes an image and calculates its Euclidean distance from all the others. Now the difference is that it keeps the 3 closest distances and looks at the classes of these images. In the end, it will select the majority of these classes (e.g., if I have 2 'cat' classes and 1 'frog', it selects 'cat').

The code for $K=3$ is located in the same code block as the one for $K=1$; we simply added the training and evaluation to the library's auxiliary commands for $K=3$ now, and at the end, we print the results.

Results for NearestNeighbor

```
Test Accuracy (K=1): 35.39%  
K=1 Training time: 91.60 seconds
```

Figure 1: Results for $K=1$

```
Test Accuracy (K=3): 33.03%  
K=3 Training time: 88.58 seconds
```

Figure 2: Results for $K=3$

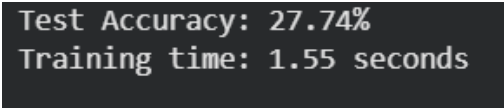
We observe that both have similar training times, around 90 seconds. However, for $K=1$ we have a 2% higher accuracy than for $K=3$. So we can easily conclude that for the entire dataset, the algorithm works better for $K=1$ than for $K=3$.

Nearest Centroid

The basic idea here is that each class is now represented by the center of its samples in the feature space. So at the beginning, the algorithm calculates the average of all the features of the samples. Then, when a new sample arrives, we calculate its distance from all these centers we have. The sample will then take the class with the smallest distance.

The code uses exactly the same logic as before; the only thing that changes now is that the library's auxiliary commands for NearestCentroid are used instead of NearestNeighbor for training and evaluation.

Results for NearestCentroid



```
Test Accuracy: 27.74%  
Training time: 1.55 seconds
```

Figure 3: Results for NearestCentroid

We see here that the training time is extremely short; however, the accuracy is quite a bit lower compared to NearestNeighbor (7% lower).

Conclusions - Comments

We can easily and objectively conclude that the best of the 3 is the 1-NearestNeighbor, as it is more efficient even if it has a slightly longer training time than the 3-NearestNeighbor. The fastest is NearestCentroid, however, due to its poor performance, it ranks last on the list. The 3-NearestNeighbor, due to its 2% lower performance, is placed below the 1-NearestNeighbor.

Neural Network Implementation for Cifar10

At this point follows the main idea of the assignment, which is the implementation of a feedforward neural network that could be either an MLP (fully connected) or a CNN (convolutional) or a combination of both. Below, there are implementations of 2 MLP models and several combination models. Before these, however, some general information about each neural network and how each one works will be mentioned. Also, some important terms necessary for understanding our models will be commented on and analyzed.

Multi Layer Perceptrons

When we say feedforward, we mean that the information moves in one direction: from the input layer, through however many hidden layers there are, and finally to the output layer, without any looping or feed-back.

MLPs are a type of feedforward neural network consisting of fully connected dense layers that transform data from one dimension to another. They are called multi-layer because they contain an input layer, zero, one, or multiple hidden layers, and an output layer.

In the hidden layer, each neuron calculates the sum of the input weights plus another bias term. Then this sum passes through an activation function to introduce the non-linearity that we want. **In our examples, we use ReLU as the activation function.** After the network produces an output, the error calculation follows through the use of a loss function. In our examples, Sparse Categorical Crossentropy is used.

After this follows the backpropagation algorithm, which minimizes the difference between the predicted and actual output. The way it works is by taking the errors backwards through the network, using the chain rule of calculus to calculate the gradients, and then recursively updating the weights and biases.

Finally, these networks rely heavily on optimization algorithms that recursively improve the weights and biases during training.

Convolutional Neural Networks

Convolutional neural networks have a slightly more complex design. They also consist of an input layer, but now the hidden layer consists of a convolutional layer, a pooling layer, and a fully connected layer (within these there is also the activation layer). At the end, classically, there is the familiar output layer as before.

At the beginning, the input layer is where, as before, we provide the input to our model, which keeps the dimensions as they are.

Then follows first the convolutional layer, in which we try to extract the shape of the image. In this layer, a set of filters are applied to our images, which are usually small matrices. What is extracted from this layer are the dimensions we had before, with the only change being the last dimension, which will now contain the number of filters.

Next, we have an activation layer in which, similar to MLPs, we introduce the activation function to the output of the previous layer. **Here too, ReLU is used in our examples.**

Then we have the pooling layer, whose main function is to reduce our first two dimensions, thus making calculations faster while simultaneously reducing memory footprint, and we can avoid overfitting. We make use of max pooling.

Before the information passes to the fully connected layer, we flatten the information into a 1D vector as it is the only way for it to enter this layer. There it makes the final necessary classification calculation before passing to the output layer.

At the output layer, we use the softmax function which converts this output we got into the probabilistic outcome of each class.

The optimization algorithms as well as the backpropagation algorithm have exactly the same logic as in MLPs.

Below follows the analysis of basic terms, as they are concepts that are used quite a lot in our models.

Basic Terms

- **Overfit:** Occurs when the model learns too much from the training data. Thus, the model works well on these data but not when tested on new data. It usually happens when the model is complex or due to the size of the training data.
- **Underfit:** Here we have the opposite of overfit, because the model cannot understand what is going on with the data. Thus the model does not work well either in training or in testing. Reasons could be the simplicity of the model, insufficient size of training data, or excessive use of regularization techniques that help prevent overfitting.
- **Epoch:** It is how many times your model will process your data. In the examples, we mostly use 100.
- **Batch size:** How many samples will be processed before our model is updated. In the examples, we mostly use 64.
- **Validation data:** It is a piece of data different from the main training set which is used to see how well the model generalizes to new data in order to appropriately adjust the training parameters.
- **Early Stopping:** It is a technique that stops training before all epochs are completed because the model is no longer improving or starts to deteriorate due to overfit.
- **Learning Rate:** It is how large the steps our network takes when updating the weights during training are. When the rate is large, the model learns quickly but can become unstable, while when it is small, the model learns more slowly but steadily.
- **Dropout:** It is a technique that randomly deactivates a percentage of neurons, thus helping to prevent overfitting. The model is therefore trained by specific neurons.
- **Batch Normalization:** A technique that normalizes the values of neurons within each batch during training. With this technique, training is stabilized, the use of a larger learning rate is allowed, and training speed is increased.
- **Weight decay or L2 Regularization:** It is also a technique that adds a small penalty to the large weights of the model so that they do not grow excessively. The goal is to reduce overfitting and stabilize the model.

These were mainly the terms that needed analysis as they are subsequently used continuously in our models. Also, this analysis of terms will greatly help in the analysis of the code that follows.

Code Analysis

We start by importing all the necessary libraries that are used later in the code. Many of the things present in our code have already been mentioned above.

We load cifar10, directly normalize the pixels, and flatten the labels to make the process easier. Then we split our data into a training set, validation set, and test set. The training set has 40,000, the validation has 10,000, and the test set remains as it was with 10,000 images.

Then the construction of our model begins, where we first get the number of our classes. Then our various layers are added depending on what we have in mind to construct. The logic of this construction is based on the information we provided earlier about the two types of neural networks we were given. In general, the examples start from simple models and as we proceed they become more complex.

Next, we compile our model using the Adam Optimizer and Sparse Categorical Crossentropy for all our examples. We also make use of Early Stopping if we observe that no improvement happens for 25 epochs, while simultaneously saving the best weights. After this, we have added ReduceLROnPlateau, which, if it sees that the model is deteriorating, slightly lowers the learning rate in order to somewhat stabilize our model.

After going through all this, we reach the point where the training of our model begins with the epochs and batch size we mentioned before. We also calculate the accuracy and error ONLY once on the entire test set.

At the end, we print all the necessary information requested by the exercise and 2 additional graphs showing the relationship between training accuracy and validation accuracy, and the relationship between training error and validation error, respectively.

As the examples go by, we will see in the code that the various regularization techniques analyzed before are added. Let's now look at our models in detail one by one.

Example-Model 1

- In this example we make use of an MLP with two hidden layers in which we place 64 neurons in each.
- We do NOT make ANY use of any regularization technique to help our model, only ReduceLROnPlateau whose function was explained above.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 212.87 seconds"
- "Test accuracy: 0.4802 , Test loss: 1.4668"

and we have the following diagrams:



Figure 4: Accuracy diagram of the 1st example

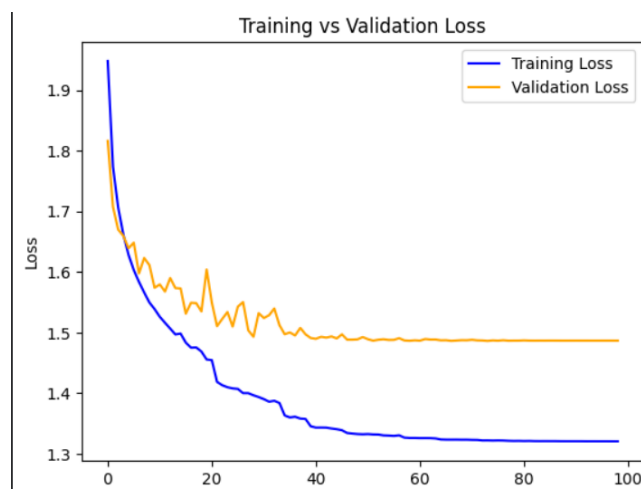


Figure 5: Error diagram of the 1st example

We see from the accuracy diagram that in the first epochs the model trains rapidly, which means it quickly learns the basic features and patterns of the data. Then the two curves rise gradually almost together up to 40 epochs. From there up to 100 epochs, a gap is observed between the two curves, and in fact both stabilize at certain values (in both of our diagrams), a fact indicating that from then on our model overfits.

The accuracy is relatively low (under 50%) because our model is relatively simple and consists of a relatively small number of neurons.

Example-Model 2

- In this example we make use of an MLP with two hidden layers in which we place 32 neurons in each this time.
- We do not make many uses of regularization techniques, only Dropout(30%) and ReduceLROnPlateau.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 191.44 seconds"
- "Test accuracy: 0.3400 , Test loss: 1.7964"

and we have the following diagrams:

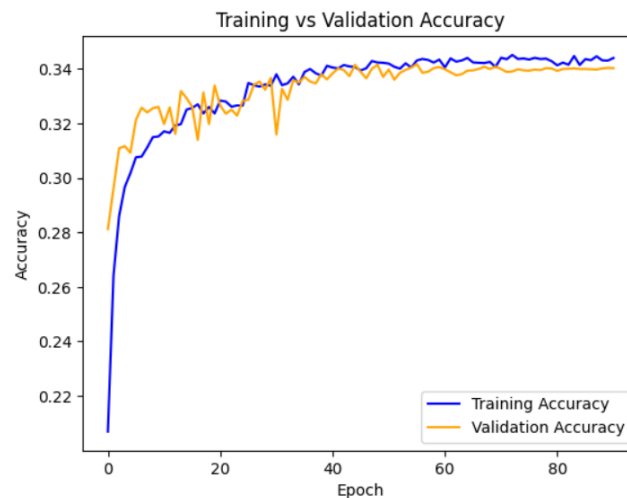


Figure 6: Accuracy diagram of the 2nd example

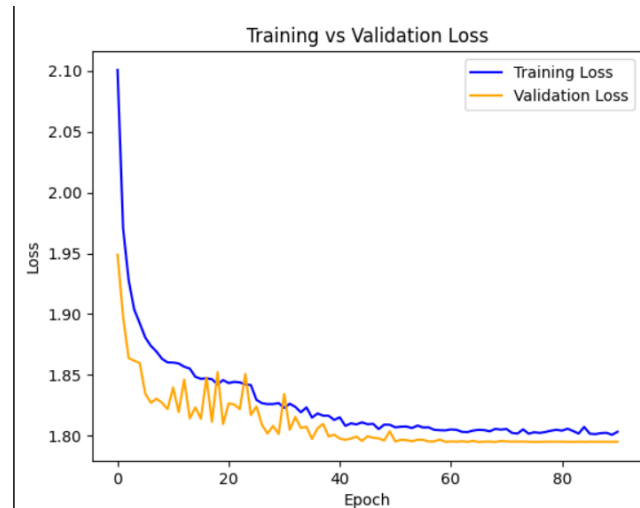


Figure 7: Error diagram of the 2nd example

Again here we see in the first 10 epochs a rapid increase in accuracy in our curves showing that our model learns the basic features quickly. Then our two curves appear to rise at a slower and steadier pace, and unlike before, they rise relatively together, meaning the curves are close to each other. Similarly and conversely, the same happens in the error curves. So we do not have overfit, but conversely, the model learns very efficiently. At the end, in the final epochs, the accuracy curves begin to stabilize around 0.34 and the error around 1.80.

We have lower accuracy than before because we use fewer neurons and also the Dropout technique simultaneously deactivates some of these neurons of ours.

Below we will see more complex models that will greatly increase accuracy, however, they will cause significant overfit. As we proceed, we will add layers and find ways to stop this overfit problem while keeping the accuracy high.

Example-Model 3

- In this example, in the hidden layer we make use of 2 Conv2D layers (2 convolutional layers), each having a Batch Normalization layer, and after all these the MaxPooling layer. The output of these meets a Fully Connected Layer of 32 neurons.
- We do not make uses of regularization techniques, only ReduceLROnPlateau.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 114.41 seconds"
- "Test accuracy: 0.6525 , Test loss: 1.0197"

and we have the following diagrams:

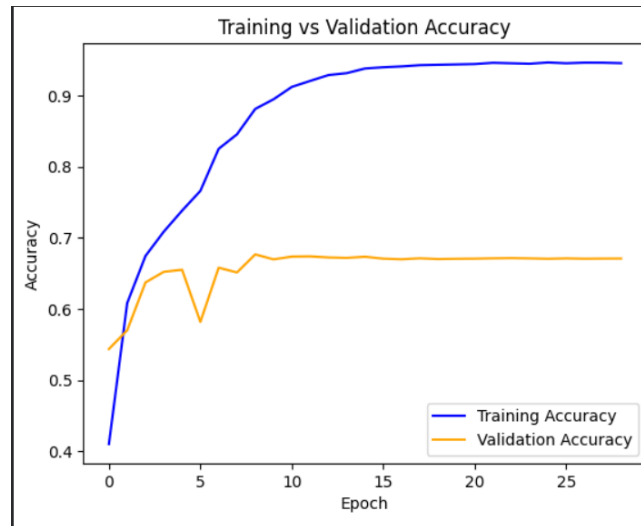


Figure 8: Accuracy diagram of the 3rd example

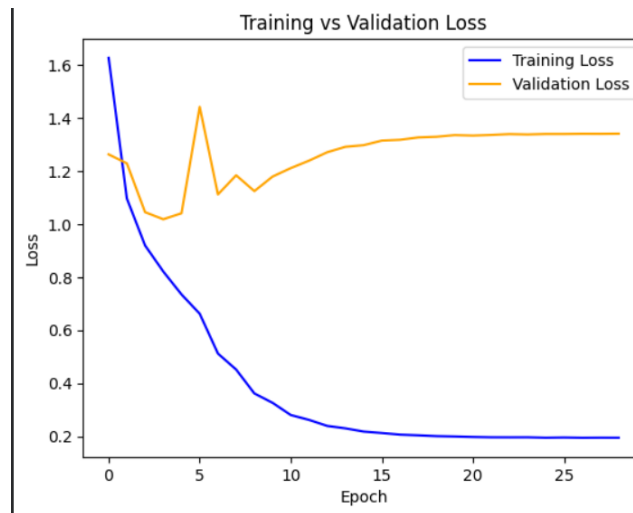


Figure 9: Error diagram of the 3rd example

We start again in accuracy with a rapid increase from about 0.4 to 0.55 in the very first epochs. The training curve goes extremely fast to around 0.9 while the validation curve goes slowly and steadily to 0.65. We observe a huge gap between the curves (either accuracy or error) which indicates severe overfit. The model stops training and somewhere around 30 epochs Early Stopping is called.

What we managed, however, compared to before, is to increase the accuracy by a very significant percentage (almost by 20%).

Example-Model 4

- In this example, in the hidden layer we make use of 4 Conv2D layers (4 convolutional layers), each having a Batch Normalization layer and 2 MaxPooling layers (one per 2 Conv2D layers). The output of these meets a Fully Connected Layer of 128 neurons.
- We do not make uses of regularization techniques, only ReduceLROnPlateau.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 168.45 seconds"
- "Test accuracy: 0.7609 , Test loss: 0.8037"

and we have the following diagrams:

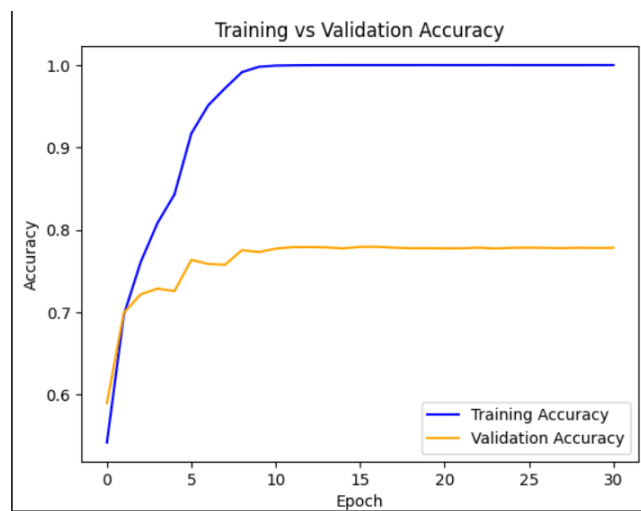


Figure 10: Accuracy diagram of the 4th example

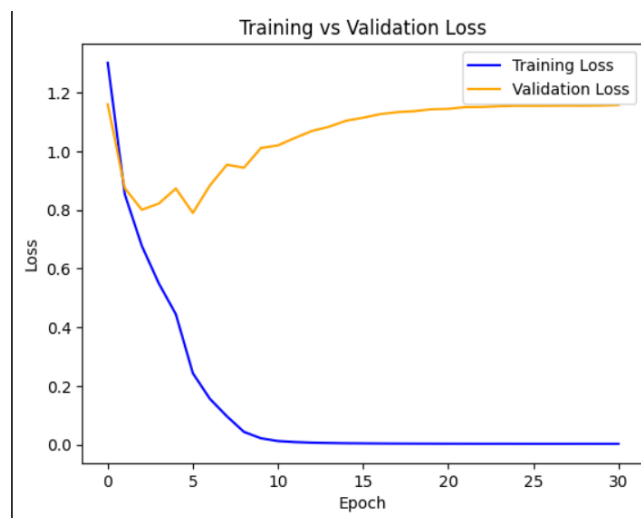


Figure 11: Error diagram of the 4th example

At the beginning, the same applies as all previous examples, however this time we go from 0.55 to about 0.7. The training curve begins to diverge very sharply already from the first epochs and manages to reach up to 1.0 (complete memorization). The other curve stabilizes somewhere at 0.77, so we have quite a bit of overfit again and for this reason Early Stopping is called somewhere around 30 epochs.

We managed however, with the addition of one block, to gain quite a lot in accuracy again by a percentage of about 10%.

Example-Model 5

- In this example, in the hidden layer we make use of 6 Conv2D layers (6 convolutional layers), each having a Batch Normalization layer and 3 MaxPooling layers (one per 2 Conv2D layers). The output of these meets a Fully Connected Layer of 256 neurons.
- We do not make use of regularization techniques, only ReduceLROnPlateau.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 190.41 seconds"
- "Test accuracy: 0.7682 , Test loss: 0.6907"

and we have the following diagrams:

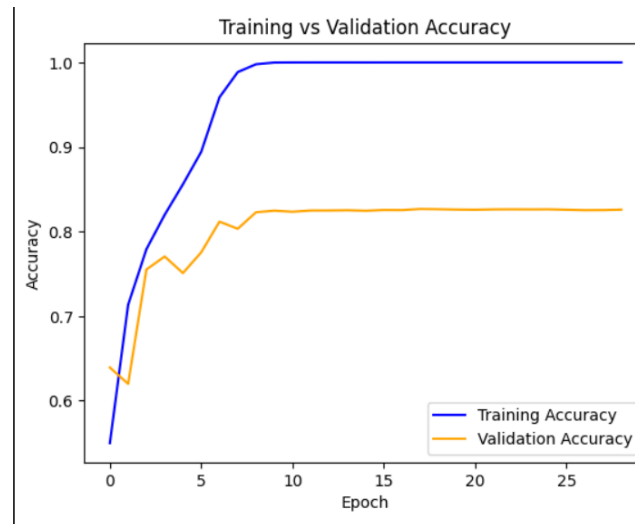


Figure 12: Accuracy diagram of the 5th example

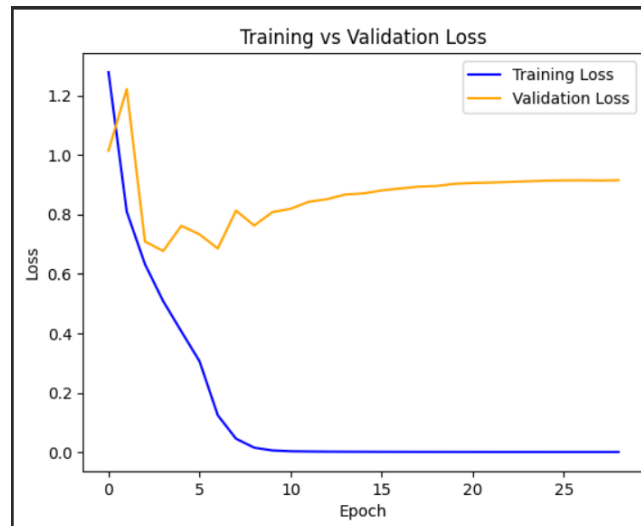


Figure 13: Error diagram of the 5th example

The situation here is exactly identical to before, so we don't have much to explain. The accuracy increased slightly and the error managed to drop by 0.1. We therefore conclude that these 6 convolutional layers (the 3 blocks together with all the other layers) are the "limit" for the model and that we don't need to make it more complex. Below, therefore, we will introduce techniques that will stop overfit while still keeping accuracy high.

Example-Model 6

- In this example, in the hidden layer we make use of 6 Conv2D layers (6 convolutional layers), each having a Batch Normalization layer and 3 MaxPooling layers (one per 2 Conv2D layers). The output of these meets a Fully Connected Layer of 256 neurons.
- Besides ReduceLROnPlateau, we now also add a Dropout(75%) before the output.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 233.17 seconds"
- "Test accuracy: 0.8100 , Test loss: 0.6199"

and we have the following diagrams:

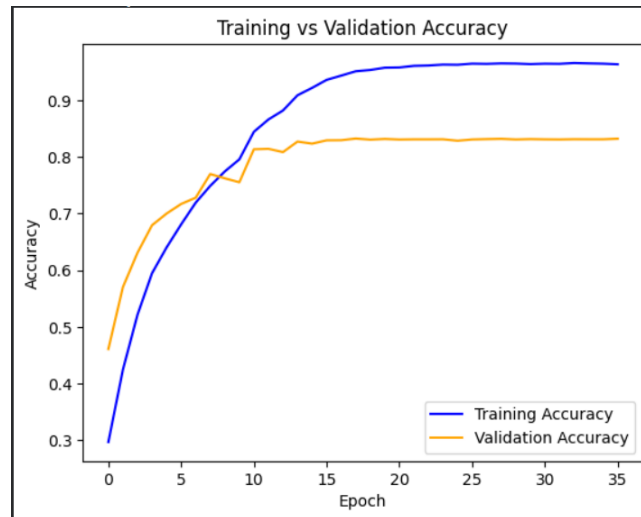


Figure 14: Accuracy diagram of the 6th example

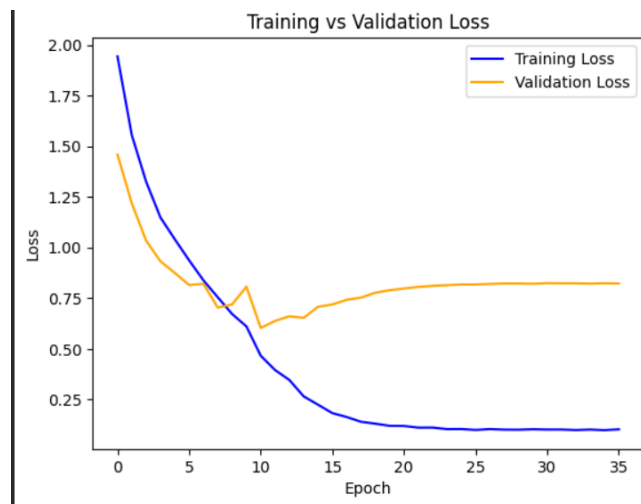


Figure 15: Error diagram of the 6th example

Things continue to be the same at the beginning and generally up to about the 10th epoch we have a good start with the model training correctly and reaching up to almost 0.78. Then there is a relatively controlled divergence compared to before, about 13%, with the training curve touching 0.95 and the validation curve 0.82. However, then there is a stabilization forcing the Early Stopping call so that the best weights are saved.

The only sure thing is that this model is definitely better than the previous ones, as we managed to touch an accuracy above 0.8. However, the overfit still remains, which means we must add something else to help our model.

Example-Model 7

- In this example, in the hidden layer we make use of 6 Conv2D layers (6 convolutional layers), each having a Batch Normalization layer and 3 MaxPooling layers (one per 2 Conv2D layers). The output of these meets a Fully Connected Layer of 256 neurons.
- We now use ReduceLROnPlateau, Dropout after each block with various values plus the Dropout that existed before (meaning a total of 4 Dropouts), and Weight Decay-L2 Regularization with $l2(1e-4)$.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 553.50 seconds"
- "Test accuracy: 0.8631 , Test loss: 0.6295"

and we have the following diagrams:

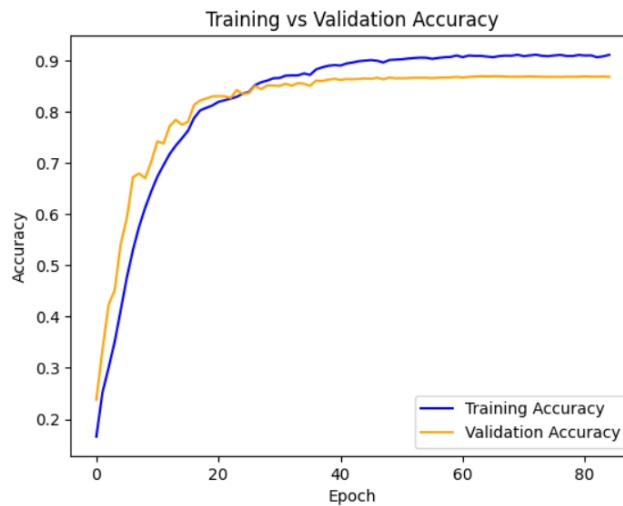


Figure 16: Accuracy diagram of the 7th example

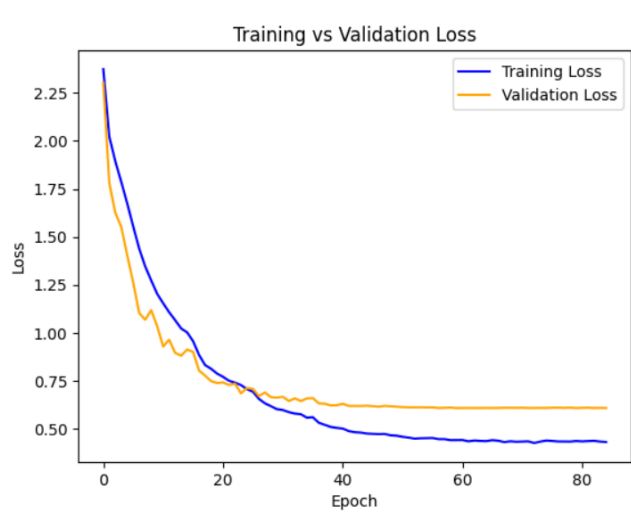


Figure 17: Error diagram of the 7th example

In this example, we have a healthy training process compared to our previous complex models. The accuracy curves at the beginning go up in the first 30 epochs up to 0.8 together (i.e., one is very close to the other, indicating that the model learns very effectively on new data). Then from there onwards the training curve stabilizes somewhere at 0.9 and the validation at 0.85, and this small difference is considered normal and acceptable. However, the model after a while ceases to improve and around 85 epochs Early Stopping occurs.

The accuracy we got is the maximum of all the examples we reached, and this is mainly due to the many auxiliary regularization techniques we used.

In the next 2 examples, we will use only one learning rate which, however, will be quite a bit smaller than the 0.001 we start with in the previous examples.

Example-Model 8

- In this example, in the hidden layer we make use of 6 Conv2D layers (6 convolutional layers), each having a Batch Normalization layer and 3 MaxPooling layers (one per 2 Conv2D layers). The output of these meets a Fully Connected Layer of 256 neurons.
- We use only a Dropout(75%) again, Weight Decay-L2 Regularization with $l2(1e-4)$ and learning rate equal to $5e-5$ this time.
- We have 100 epochs, 64 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 369.74 seconds"
- "Test accuracy: 0.7253 , Test loss: 0.9863"

and we have the following diagrams:

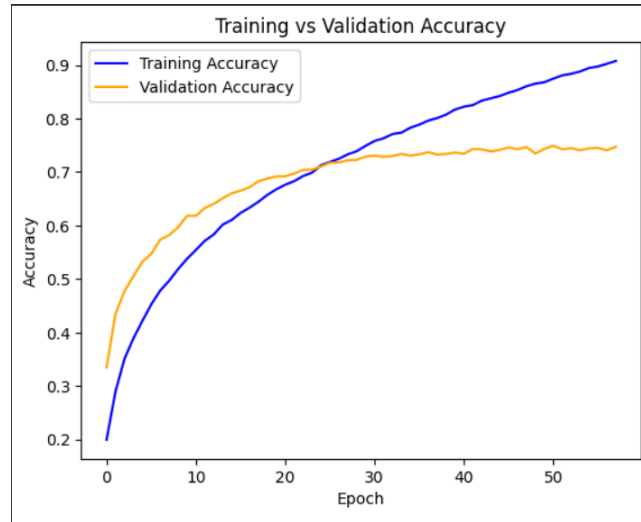


Figure 18: Accuracy diagram of the 8th example

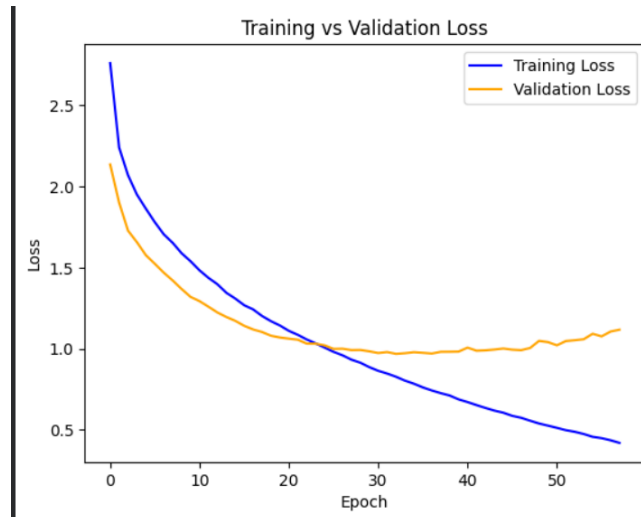


Figure 19: Error diagram of the 8th example

We see how due to the small learning rate we set, the validation curve for accuracy is initially above the training one. Also, the use of Dropout reduces the training score and for this reason it is lower. Then the training curve passes from above and reaches up to 0.9, while our other curve we see stabilizes around 0.7. We have, that is, overfit again.

The accuracy is logical to be lower since the learning rate is now relatively low.

Example-Model 9

- In this example, in the hidden layer we make use of 6 Conv2D layers (6 convolutional layers), each having a Batch Normalization layer and 3 MaxPooling layers (one per 2 Conv2D layers). The output of these meets a Fully Connected Layer of 256 neurons.
- We use only a Dropout(75%) again, Weight Decay-L2 Regularization with $l2(1e-3)$ and learning rate equal to $1e-5$ this time.

- We have 150 epochs, 32 batch size, and Adam Optimizer.

Our code prints:

- "Total training time = 833.35 seconds"
- "Test accuracy: 0.6884 , Test loss: 1.5575"

and we have the following diagrams:

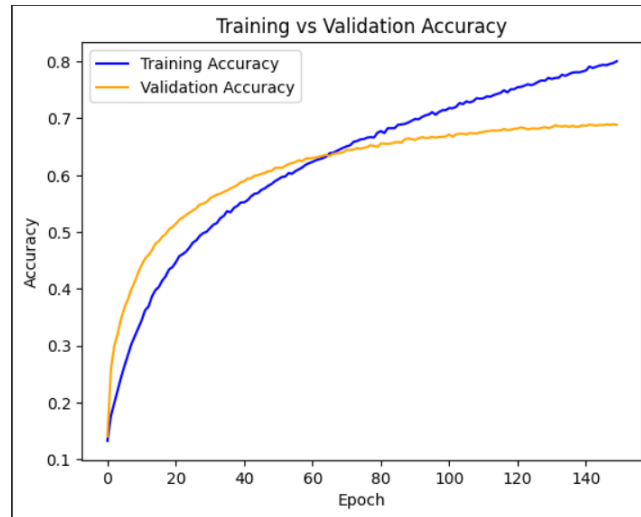


Figure 20: Accuracy diagram of the 9th example

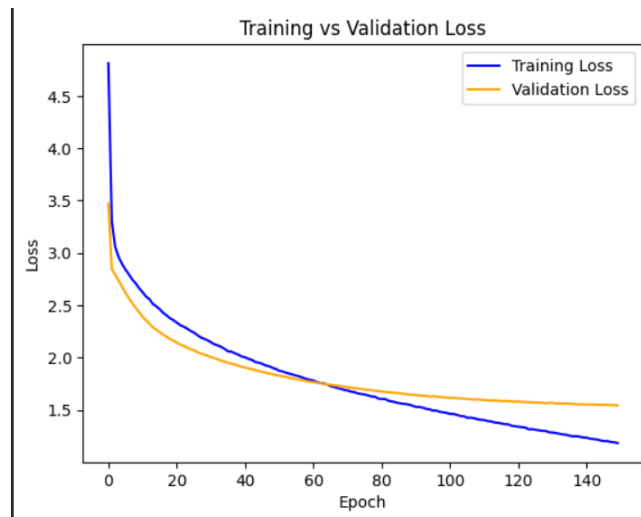


Figure 21: Error diagram of the 9th example

Unlike all previous examples, here we observe the phenomenon of underfit. This is observed in the accuracy diagram from the fact that the validation curve is above the training curve for 60 epochs. Then the training curve surpasses that of validation and continues its upward gradual course, while the other stabilizes somewhere at 0.65 with no signs of improvement. Then at the end there, we can consider that we have a mild form of overfit. The accuracy is logical to be so low since we reduced the learning rate to an excessive degree ($1e-5$).

Categorization Examples

Below we have three categorization examples; the first two concern a weak model, where it first predicts correctly and then incorrectly. In the third, a better model takes the same image and we see it predict correctly now compared to before. The code loads the model we want to test, then we print the image we are testing, and then through the tensorflow functions, we produce the result and print it.



Figure 22: Correct classification/2nd model

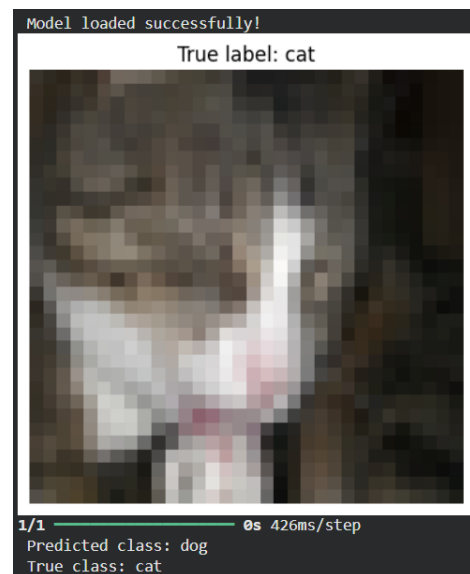


Figure 23: Incorrect classification/2nd model

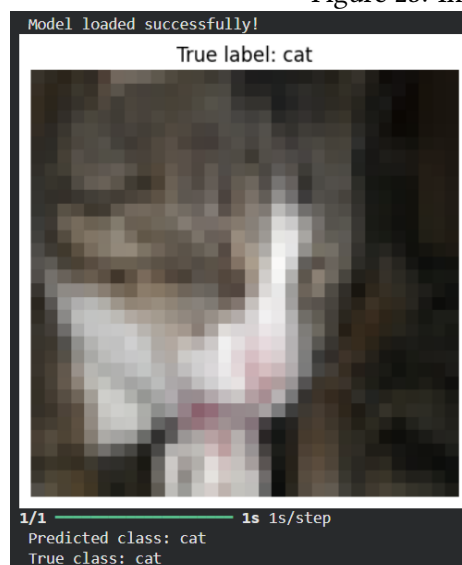


Figure 24: Correct classification/7th model

Final Comments for the Entire Assignment

- We observe that our MLP models present relatively smooth training, yet low accuracy.
- The simpler our model is, the lower the accuracy will be.
- For this reason, it is also possible to see accuracy lower than even the KNN and NC algorithms that we used at the beginning.
- In general, however, the implementation of a neural network is considered better, especially when we have complex data like Cifar10.
- Our CNN models, although they achieve greater accuracy (especially as they become more complex), present the phenomenon of overfit quite a bit.
- We concluded that the limit of convolutional layers was 6 (or 3 blocks) as we did not observe further improvement.
- The purpose then was, with these layers, to significantly solve the overfit phenomenon while keeping accuracy high, which we managed with the various regularization techniques.
- Our best model managed to achieve healthy training and reach 0.86 accuracy.
- Finally, we saw how the use of a lower learning rate, while eliminating overfit, also drops our accuracy significantly.